Data Consistency: Deleting with Foreign Keys

Lecture Notes · Game DB Junior

In this lesson

- What happens when we delete a record that other rows **depend on**
- **Dangling references**
- The five **ON DELETE** options

1. The problem

We saw how inserting works with foreign keys. Now we look at the opposite: **deleting** a record that involves a foreign key.

In the photos table, three photos (ids 2, 3, 4) belong to the user with **id 1** (monahan93). What happens if we delete this user?

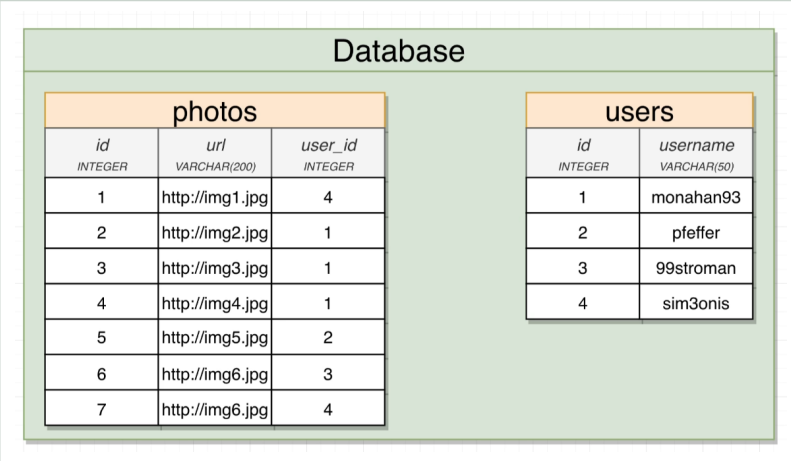


Figure 1 — Photos 2, 3 and 4 belong to user 1

2. Dangling references

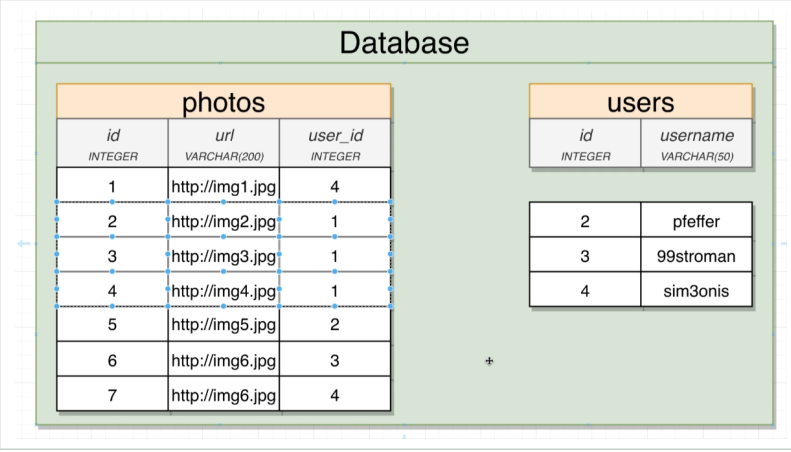


Figure 2 — User 1 is deleted, but photos 2, 3, 4 still point at it

- If user 1 is deleted, the three photos point at a user that **does not exist**. These are **dangling references**.
- The ids use **SERIAL**. With SERIAL, an id is **never reused**, even after the record is deleted. So there will never be another user with id 1.
- These photos will never point at any user again. We would have to write code to find them, which is annoying.
- So with foreign keys, we can choose **what should happen** when we delete a record that other rows depend on. Here, the three photos **depend on** user 1.

3. The five ON DELETE options

Option	What happens if we delete a user that a photo still references
ON DELETE RESTRICT	Throw an error. This is the **default**, used if we choose nothing
ON DELETE NO ACTION	Throw an error. There is a very small difference from RESTRICT; we learn it later
ON DELETE CASCADE	Delete the photos too
ON DELETE SET NULL	Set the user_id of the photos to NULL
ON DELETE SET DEFAULT	Set the user_id of the photos to a default value, if one is given

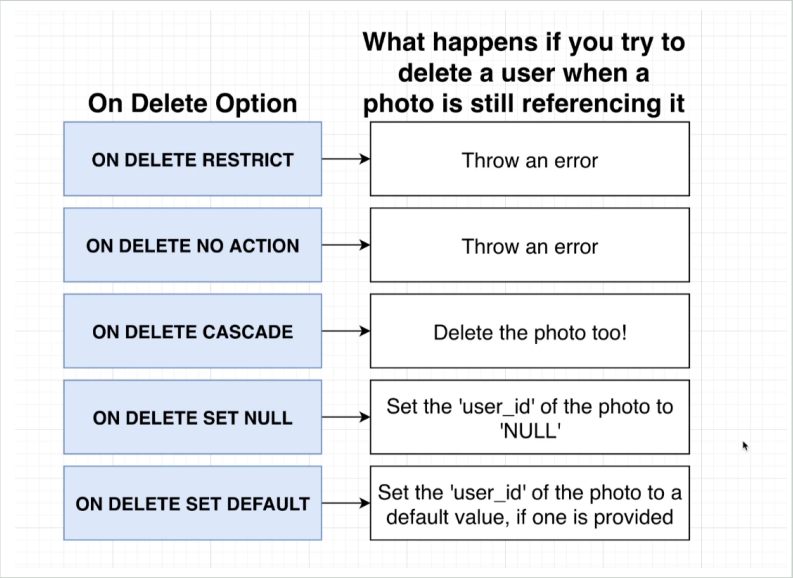


Figure 3 — What happens if you delete a user when a photo still references it

ON DELETE RESTRICT (the default)

If we try to delete user 1, we get an error. This foreign key constraint stops the delete.

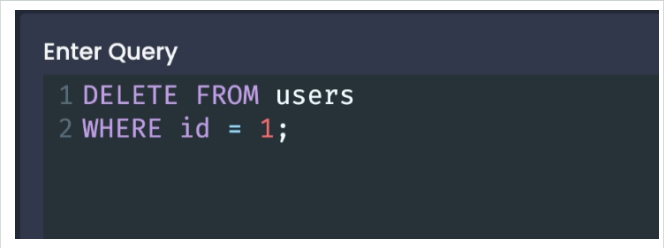


Figure 4 — DELETE FROM users WHERE id = 1

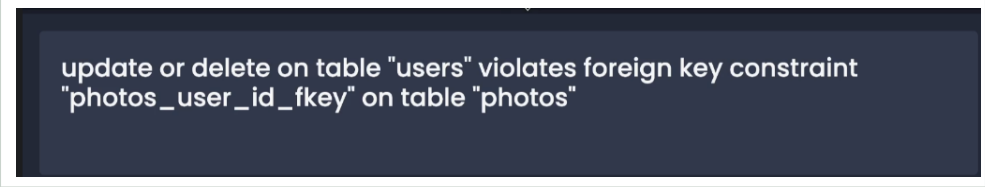


Figure 5 — Error: violates foreign key constraint

ON DELETE CASCADE

- When we delete a user, Postgres also finds and deletes **all the photos** that point at it.
- This is often a really good choice if we want to delete a user **and all their data**.

ON DELETE SET NULL

- When we delete user 1, the user_id of photos 2, 3 and 4 becomes **NULL**.
- The photos still exist, but it is clear that **no user** is tied to them.

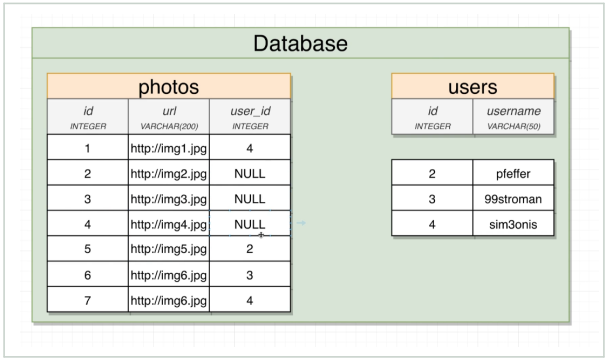


Figure 6 — SET NULL: photos 2, 3, 4 now have user_id NULL

ON DELETE SET DEFAULT

- When we create a table, a column can have a **default value**. If we insert a row and give no value for that column, the default value is used.
- With this option, if the user is deleted, the photo's user_id gets that default value. We will see an example later.

4. Summary

Idea	Remember
Dangling reference	A foreign key that points at a record that no longer exists
SERIAL ids	Never reused, even after a delete
RESTRICT / NO ACTION	Error. RESTRICT is the default
CASCADE	Delete the related rows too
SET NULL / SET DEFAULT	Keep the rows, set the foreign key to NULL or a default value

Next lesson: where we set these options, and tests with CASCADE and SET NULL.

Sources: Lecture transcript and lecture images (Figures 1–6)